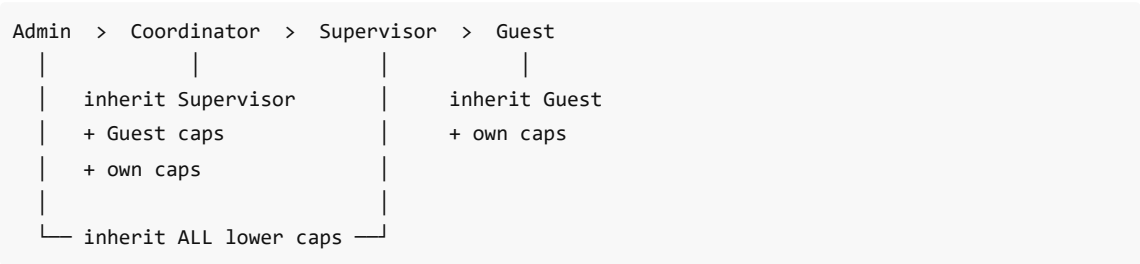


PBAC Matrix — Inspect-able

Policy-Based Access Control. Roles define administrative scope; Qualifications grant operational inspection authority. The engine unions capabilities from both.

1. Role Hierarchy



Higher roles inherit **all** capabilities of lower roles. You never need multiple roles like `Coordinator + Supervisor` .

2. Administrative Role → Capability Matrix

Capability	Guest	Supervisor	Coordinator	Admin
view:dashboard	✓	✓ (inherited)	✓ (inherited)	✓ (inherited)
manage:locations	—	✓ (dept-scoped)	✓ (inherited)	✓
schedule:manage_dept	—	✓	✓ (inherited)	✓ (inherited; covered by schedule:manage_all)
schedule:manage_all	—	—	—	✓
assign:others	—	—	✓	✓ (inherited)
view:all_departments	—	—	✓	✓ (inherited)
manage:departments	—	—	✓ (dept-scoped)	✓
manage:users	—	—	✓ (dept-scoped)	✓
manage:groups	—	—	✓	✓ (inherited)
manage:mappings	—	—	✓	✓ (inherited)
manage:kpi	—	—	—	✓
manage:phases	—	—	—	✓
manage:permissions	—	—	—	✓
manage:settings	—	—	—	✓

manage:certs	—	—	—	✓
purge:data	—	—	—	✓
system:admin	—	—	—	✓
system:reset	—	—	—	✓

3. Qualification → Capability Matrix

Qualification	Activation	Capabilities Granted
Inspector	qualifications[] contains "Inspector"	asset_inspector, assign:self

Note: Inspector qualification grants the `asset_inspector` and `assign:self` capabilities. However, a valid certificate (`certificationExpiry ≥ today`) is **also required** by the `CERT_VALID` policy within `CanInspectAudit` to actually perform inspection actions. Qualification and certificate work together — neither alone is sufficient for inspection eligibility.

4. Role + Qualification → Combined Capabilities

Administrative Role	Qualification	Combined Capabilities (Union)
Admin	—	Full system administration
Admin	Inspector	Full admin + <code>asset_inspector</code> , <code>assign:self</code>
Coordinator	—	Manage own department, assign others, manage users/locations/groups/mappings
Coordinator	Inspector	Dept management + self-assign to cross-dept audits
Supervisor	—	Manage locations in dept, manage dept schedules
Supervisor	Inspector	Location management + self-assign as inspector
Guest	—	View dashboard only
Guest	Inspector	Dashboard view + self-assign as inspector

Design principle: Never store `Coordinator + Inspector` as a role. Store `{ role: "Coordinator", qualifications: ["Inspector"] }` and let the policy engine derive the combined capability set.

5. Action → Policy Mapping

5.1 Audit Schedule Actions

Action	Policies Applied	Who Can Perform
schedule.assign	CAN_SELF_ASSIGN	Users with <code>asset_inspector</code> + <code>assign:self</code>

schedule.unassign	SLOT_OWNER_OR_PRIVILEGED	Slot owner or privileged role
schedule.lock	schedule:manage_dept OR schedule:manage_all	Supervisor / Admin
schedule.set_date	(schedule:manage_dept OR schedule:manage_all) + DATE_WITHIN_PHASE	Supervisor / Admin
schedule.set_status	(schedule:manage_dept OR schedule:manage_all) + VALID_STATUS_TRANSITION	Supervisor / Admin
schedule.upload_report	ASSIGNED_AUDITOR_ONLY + CERT_VALID	Assigned auditor (certificate must be valid at upload time)

5.2 Audit CRUD

Action	Policies Applied
audit.create	assign:others + COORDINATOR_DEPT_SCOPE
audit.delete	assign:others + COORDINATOR_DEPT_SCOPE
audit.maintenance	manage:departments + COORDINATOR_DEPT_SCOPE

5.3 User Management

Action	Policies Applied
user.create	manage:users + COORDINATOR_DEPT_SCOPE
user.update	Self-update OR manage:users
user.delete	manage:users
user.verify	manage:users
user.certify	manage:certs (Admin only)

5.4 Administration

Action	Required Capability
admin.manage	system:admin
department.manage	manage:departments
location.manage	manage:locations
group.manage	manage:groups
kpi.manage	manage:kpi
phase.manage	manage:phases
permission.manage	manage:permissions
mapping.manage	manage:mappings

system.settings	manage:settings
system.reset	system:reset
data.purge	purge:data

6. CanInspectAudit — Inspection Eligibility Rules

These six policies together define who is eligible to be assigned as an inspector:

CanInspectAudit:	
1. user.qualifications contains "Inspector"	→ REQUIRE_INSPECTOR
2. user.certificate is valid (not expired)	→ CERT_VALID
3. audit.department != user.department	→ STRICT_COI
4. user is not the site supervisor of this loc	→ NO_SUPERVISOR_CONFLICT
5. schedule date falls within any configured phase	→ DATE_WITHIN_PHASE
6. location not already inspected in scheduleDate year	→ NO_ANNUAL_CONFLICT

#	Policy	Denial Reason Code	Denial Message
1	REQUIRE_INSPECTOR	MISSING_CAPABILITY	Access Denied: Inspector qualification is required for this operation.
2	CERT_VALID	CERT_EXPIRED	Your Inspector certificate is expired or invalid.
3	STRICT_COI	COI_VIOLATION	You cannot audit your own department.
4	NO_SUPERVISOR_CONFLICT	SUPERVISOR_CONFLICT	You are a Site Supervisor for this location and cannot act as its inspector.
5	DATE_WITHIN_PHASE	DATE_OUTSIDE_PHASE	The scheduled date does not fall within any configured audit phase.
6	NO_ANNUAL_CONFLICT	LOCATION_YEAR_CONFLICT	This location is already scheduled to be inspected in the calendar year of the scheduled date.

STRICT_COI has no exemptions — not even for Admins. It is an absolute institutional integrity rule.

7. Policy Definitions Reference

Core Policies

Policy	Type	Description
REQUIRE_INSPECTOR	Capability gate	User's capabilities must contain <code>asset_inspector</code> (derived from <code>qualifications[]</code> by the policy engine)
CERT_VALID	Expiry gate	<code>certificationExpiry</code> must be present and $\geq$ current date in the organization's timezone (Asia/Kuala_Lumpur)
STRICT_COI	Integrity rule	<code>user.departmentId</code> $\neq$ <code>targetDepartmentId</code> — no exemptions

NO_SUPERVISOR_CONFLICT	Integrity rule	User must not be listed as supervisor for the target location
DATE_WITHIN_PHASE	Phase scheduling rule	<code>scheduleDate</code> must fall within ANY configured audit phase (Phase 1, 2, or 3). Allows planning ahead across all phases. If <code>scheduleDate</code> is missing or no phase contains the date, deny with <code>DATE_OUTSIDE_PHASE</code>
NO_ANNUAL_CONFLICT	Scheduling rule	Location must not have an active or completed inspection in the calendar year of <code>scheduleDate</code> ; cancelled/deleted schedules are ignored. When updating an existing schedule, the current record's own <code>scheduleId</code> / <code>auditId</code> is excluded from the conflict check
NO_DOUBLE_BOOKING	Concurrency gate	Schedule slot must be 'open'

Composite Policies

Composite	Constituent Policies	Used By
CAN_INSPECT_AUDIT_POLICIES	REQUIRE_INSPECTOR + CERT_VALID + STRICT_COI + NO_SUPERVISOR_CONFLICT + DATE_WITHIN_PHASE + NO_ANNUAL_CONFLICT	<code>auditAssignmentGuard</code> (all <code>CanInspectAudit</code> checks)
CAN_SELF_ASSIGN	CAN_INSPECT_AUDIT_POLICIES + <code>assign:self</code> + NO_DOUBLE_BOOKING	<code>schedule.assign</code> (self-assignment to an open audit slot)
CAN_ASSIGN_OTHER_INSPECTOR	Actor must hold <code>assign:others</code> ; actor can manage the audit's department (<code>COORDINATOR_DEPT_SCOPE</code>); target inspector passes <code>CAN_INSPECT_AUDIT_POLICIES</code> for that audit	Coordinator/Admin assigning another inspector

Structural Policies

Policy	Description
COORDINATOR_DEPT_SCOPE	Coordinators can only act within their own department (Admin bypasses)
CAN_UPDATE_USER	Self-update always allowed; updating others requires <code>manage:users</code>
CAN_ASSIGN_OTHERS	Requires <code>assign:others</code> capability (Admin / Coordinator)
SLOT_OWNER_OR_PRIVILEGED	Only the assigned slot owner or a privileged role (Supervisor/Coordinator/Admin) may unassign
ASSIGNED_AUDITOR_ONLY	Only the auditor assigned to the slot may upload the inspection report
VALID_STATUS_TRANSITION	Status changes must follow allowed transitions: <code>open</code> → <code>assigned</code> → <code>in_progress</code> → <code>submitted</code> → <code>reviewed</code> → <code>closed</code> . Arbitrary jumps are denied. Admin may override with <code>system:admin</code> capability — override must be logged with <code>auditLogReason</code>
REQUIRE_CAPABILITY(<i>cap</i>)	Generic factory — denies unless user holds a specific capability string

8. Data Model

User Record (relevant fields)

Field	Type	Purpose
roles	string[] (JSON)	Administrative role hierarchy: ["Admin"], ["Coordinator"], ["Supervisor"], ["Guest"]
qualifications	string[] (JSON)	Operational qualifications: e.g. ["Inspector"]
certificationExpiry	string (ISO date)	Institutional certificate expiry; required by CERT_VALID to activate inspection actions
departmentId	string	User's home department (for COI checks)

Evaluation Context (per-request)

Field	Source	Purpose
targetDepartmentId	Audit's department_id	COI: compare against user's department
supervisorIds	Audit's supervisor_id (comma-separated)	Supervisor conflict check
scheduleDate	Audit schedule date	Date being assigned or updated; used by DATE_WITHIN_PHASE and NO_ANNUAL_CONFLICT
phaseId	Selected audit phase	Identifies Phase 1 / Phase 2 / Phase 3
dateInAnyPhase	Pre-computed by handler	True if scheduleDate falls within ANY configured phase boundary; used by DATE_WITHIN_PHASE
scheduleStatus	Audit's current slot status	Double-booking prevention
currentStatus	Current schedule status value	Used by VALID_STATUS_TRANSITION to validate allowed transitions
nextStatus	Target status to transition to	Used by VALID_STATUS_TRANSITION to validate allowed transitions
currentScheduleId	Current schedule record ID (if updating)	Excluded from NO_ANNUAL_CONFLICT to prevent self-conflict on edit
currentAuditId	Current audit record ID (if updating)	Alternative identifier excluded from NO_ANNUAL_CONFLICT when conflict check uses audit records
auditLogReason	Request body / admin override form	Required when system:admin overrides an invalid status transition; written to audit log
hasAnnualConflict	DB query (checkLocationYearConflict)	Annual inspection limit for scheduleDate's calendar year

9. Key Design Principles

1. **Roles are hierarchical.** Higher roles inherit all lower-role capabilities.
2. **Qualifications are orthogonal.** Inspector is a qualification, never a role — stored in `qualifications[]`, not `roles[]`.
3. **PBAC unions capabilities.** `deriveCapabilities() = deriveRoleCapabilities() ∪ deriveQualificationCapabilities()`.
4. **Policies are pure functions.** Each returns `{ allowed, reason? }`; first DENY short-circuits.
5. **STRICT_COI has no exemptions.** Even Admins cannot audit their own department.
6. **Never check role strings in UI code.** Always use `hasCapability(user, 'system:admin')`.
7. **Avoid role explosion.** Store `{ role: "Coordinator", qualifications: ["Inspector"] }`, never `"Coordinator+Inspector"`.